

Final Project Proposal: Scalable Deep Learning and EM Clustering for Single-Cell Transcriptomics

Shuai Yuan, YINUO Zhang, Yuanyuan Yan, Jinwei Zhang

1 Problem Statement

Understanding cellular heterogeneity is a fundamental challenge in modern biomedical research. Single-cell RNA sequencing (scRNA-seq) provides high-resolution gene expression profiles of individual cells, but the resulting datasets are massive, highly sparse, and inherently noisy. The objective of this project is to develop a highly scalable computational pipeline that utilizes deep learning to denoise and compress this biological data, followed by rigorous statistical clustering to accurately identify distinct cell populations.

2 Background

Traditional linear dimensionality reduction methods, such as Principal Component Analysis (PCA), often fail to capture the complex, non-linear manifolds characteristic of single-cell gene expression. Recent advancements demonstrate that deep learning architectures such as autoencoders provide superior low-dimensional representations. Once projected into a latent space, probabilistic models like Gaussian Mixture Models (GMMs) can effectively cluster cells into biological subtypes. This project bridges modern artificial intelligence with classic statistical optimization methods.

3 Data

We will utilize the **Tabula Muris Single-Cell Transcriptome Data (Mouse Brain Subset)**, accessible via Kaggle. The dataset contains expression levels for thousands of genes across nearly 100,000 cells, represented as a large, sparse matrix. Data preprocessing will require out-of-core computing and sparse matrix representations (e.g., CSR/CSC formats). Preprocessing steps will include library-size normalization, log-transformation, and the selection of the top 2,000 highly variable genes (HVGs) to mitigate noise and reduce the initial feature space.

4 Proposed Approach

Our methodology integrates three core computational modules:

- **Deep Learning:** We will implement a deep Autoencoder using PyTorch to compress the sparse, high-dimensional gene expression vectors into a dense, low-dimensional latent space.
- **Optimization (EM Algorithm):** To identify cell clusters, we will implement a GMM. We will write the Expectation-Maximization (EM) algorithm from scratch to estimate the parameters (means, covariances, and mixing coefficients) of the underlying distributions.
- **Statistical Computing & Scalability:** We will benchmark the computational complexity of the EM algorithm, focusing on how $\mathcal{O}(N \cdot K \cdot D^2)$ scales with the number of cells (N), clusters (K), and latent dimensions (D). We will leverage vectorization and GPU acceleration to ensure efficient convergence.

5 Evaluation Plan

We will evaluate our pipeline across three dimensions:

- **Statistical Validation:** Monitoring the convergence of the EM algorithm by tracking the log-likelihood over iterations.
- **Biological Utility:** Assessing the quality and separation of the cell clusters using the Silhouette Score and the Davies-Bouldin index.
- **Computational Efficiency:** Profiling execution time and memory consumption, systematically comparing standard Python loops against highly vectorized NumPy and PyTorch GPU frameworks.

6 Timeline

- **Week 8:** Acquire dataset, implement sparse matrix data structures, and complete preprocessing pipeline.
- **Week 10:** Develop, train, and hyperparameter-tune the Deep Autoencoder in PyTorch.
- **Week 12:** Implement the EM algorithm for GMM clustering from scratch; conduct computational benchmarking.
- **Week 14:** Finalize evaluations, generate latent space visualizations (e.g., UMAP), and compile the final report.